UNIVERSIDAD EAFIT | Inspira Crea Transforma

ENTREGABLE 3

El corazón del proyecto

Fallas configurables · Experimentación · Observabilidad

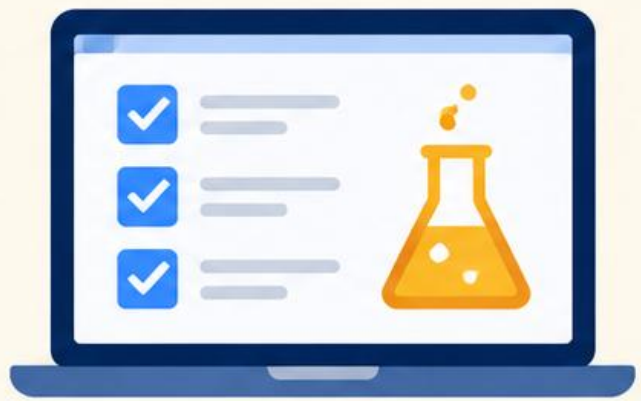
1 Configura las fallas

4 familias, 1–2 fallas por familia.

- Latencia
- Base de datos
- Degradación
- Disponibilidad

2 Experimenta

Ejecuta 4–8 experimentos (1–2 fallas por familia).



3 Observa y explica

Analiza qué pasa, cómo detectarlo, por qué ocurre y qué decisión tomar.



PRINCIPIO

La falla no es el resultado.
El resultado es lo que aprendemos al observar la respuesta del sistema.

UNIVERSIDAD EAFIT | Inspira Crea Transforma

CONDICIÓN FUNDAMENTAL

¿Qué significa “falla configurable”?

Toda falla debe cumplir cinco condiciones.



Sin redespiegue

Archivo, variable o endpoint administrativo.



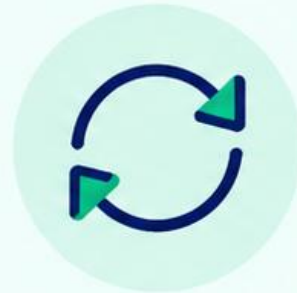
Parametrizada

Intensidad, probabilidad, alcance, duración.



Focalizable

Endpoint, módulo o % del tráfico cuando aplique.



Reversible

Kill switch durante la demostración.



Reproducible

Misma configuración + semilla → comportamiento comparable.



Una falla que solo lanza una excepción no cumple la definición de falla configurable.

UNIVERSIDAD EAFIT | Inspira Crea Transforma

CATÁLOGO

Cuatro familias · 1–2 fallas por familia

No se compensa la ausencia de una familia con más fallas de otra.

A Latencia inducida

- Latencia fija
- Jitter
- Pérdida/timeout
- Latencia parcial



B Falla de base de datos

- Caída temporal
- Pool agotado
- Bloqueo
- Failover



C Degradación de tiempos

- CPU
- Memoria/GC
- Pool de hilos
- Consumidor pausado



D Riesgo de disponibilidad

- Instancia caída
- Partición de red
- DNS
- 5xx externos
- Reloj





El objetivo es experimentar, observar y aprender.

Una falla que solo lanza una excepción no cumple la definición.

UNIVERSIDAD EAFIT | Inspira Crea Transforma

OBSERVABILIDAD

Descubrir qué señales permiten entender la falla

No se evalúa tener muchas herramientas; se evalúa su utilidad.

Métricas



Rendimiento y recursos del sistema.

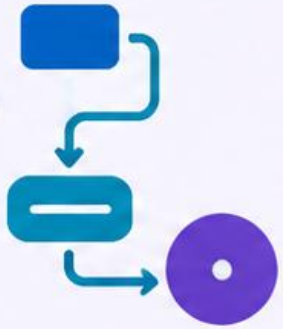
- p50/p95/p99
- CPU
- Memoria
- GC / colas

Logs



Qué ocurrió y en qué componente.

Trazas



Cómo se propaga la solicitud en componentes.



EL EQUIPO DEBE PODER RESPONDER

1. ¿Qué ocurrió?
2. ¿Dónde está la falla?
3. ¿Qué la está causando?
4. ¿Qué señales fueron útiles?
5. ¿Cuáles no lo fueron?

UNIVERSIDAD EAFIT | Inspira Crea Transforma

APRENDIZAJE

¿Qué deben descubrir sobre observabilidad?



Ventajas

Detectar degradación, localizar problemas y comparar antes/después.



Desventajas

Costo, ruido, overhead y falsos positivos.



Usos posibles

Explicar incidentes, mejorar SLO/SLA y orientar decisiones.

UNIVERSIDAD EAFIT | Inspira Crea Transforma

RÚBRICA

Evaluación · 100 puntos

La observabilidad y el método experimental concentran el mayor peso.

Fallas configurables	20%
Diseño experimental y predicciones	20%
Ejecución, reproducibilidad y reversibilidad	15%
Análisis de resultados y aprendizaje	10%
Observabilidad y evidencia	25%
Corrección, reexperimentación y comunicación	10%

Observabilidad y evidencia = 25%

Criterio de mayor peso

¿Qué debe quedar en el repositorio?



Configuración

Catálogo, parámetros, targets y switches.



Código

Base → falla → medición → corrección → repetición.



Evidencia

Experimentos, resultados, logs y métricas.



Informe

Máximo 10 páginas + anexos técnicos.



Tabla obligatoria:

ID · familia · falla · configuración · predicción · resultado · evidencia · aprendizaje

Condiciones mínimas

Además de la nota de la rúbrica, todas deben cumplirse.



4–8 experimentos
1–2 por familia.



Cuatro familias
con fallas configurables.



Activación sin redepliegue.



Predicción previa documentada.



Procedimiento reproducible.



Kill switch funcional.



Evidencia observable
por experimento.



Análisis y aprendizaje
de cada experimento.

Una demostración de falla no es un experimento



Gráficas sin hipótesis ni contexto.



“La aplicación se cayó” como único resultado.



Muchos dashboards sin interpretación.



Promedios como única medición de latencia.



La diferencia entre la predicción y el resultado es la nota.

UNIVERSIDAD EAFIT

ENTREGABLE 3

El corazón del proyecto

Fallas configurables, experimentación y observabilidad

Aplicaciones y Sistemas Escalables

Entender. Crear. Transformar.

1. Objetivo

Construir un laboratorio de fallas configurable y reproducible sobre el sistema desarrollado, para entender qué ocurre cuando falla, cómo detectarlo con observabilidad y qué decisiones de diseño tomar.

2. Fallas configurables

Cada falla debe cumplir las cinco condiciones:

1. **Sin redespliegue** — Se activa por archivo, variable o endpoint admin.
2. **Parametrizada** — Intensidad, probabilidad, alcance, duración.
3. **Focalizable** — Endpoint, componente o % de tráfico.
4. **Reversible en segundos** — Kill switch durante la demostración.
5. **Reproducible** — Misma config. + misma semilla = mismo comportamiento.

3. Cuatro familias de fallas (mínimo dos por familia)

A · Latencia inducida
- Latencia fija en dependencia
- Jitter (distribución lognormal)
- Solo en parte del tráfico
- Timeout controlado

B · Falla de base de datos
- Caída temporal de PostgreSQL
- Pool de conexiones agotado
- Bloqueo sobre un recurso
- Failover o disco lleno simulado

C · Degradación de tiempos
- Saturación de CPU
- Presión de memoria y GC
- Inanición del pool de hilos
- Consumidor pausado / lag

D · Riesgo de disponibilidad
- Caída de una instancia
- Partición de red servicio ↔ base
- DNS que no resuelve
- Respuestas 5xx externas
- Desfase de reloj

4. Método experimental

Cada falla se documenta con la predicción escrita antes de conocer el resultado.

1. **Estado base** — Métricas de referencia sin falla.
2. **Predicción (antes)** — Qué se espera que ocurra y por qué.
3. **Activar falla** — Con una configuración especifica.
4. **Observar y medir** — Métricas, logs y/o trazas.
5. **Explicar** — Comparar predicción vs. realidad y analizar diferencias.
6. **Corregir y repetir** — Ajustar configuración y reejecutar para validar el efecto.

5. Observabilidad

Descubrir qué señales permiten entender la falla.

Métricas
p50/p95/p99, throughput, errores, CPU, memoria, GC, colas, circuit breakers.

Logs
Reconstruir qué ocurrió y cuándo.

Trazas
Seguir una solicitud a través de componentes y dependencias.

Y responder:
- ¿Qué ocurrió?
- ¿Dónde está la falla?
- ¿Cuáles no lo fueron?
- ¿Qué la está causando?
- ¿Qué señales fueron útiles?

6. Rúbrica de evaluación (100 puntos)

Criterio	Peso
1. Fallas configurables	20%
2. Diseño experimental y predicciones	20%
3. Ejecución, reproducibilidad y reversibilidad	15%
4. Observabilidad y evidencia	25%
5. Análisis de resultados y aprendizaje	10%
6. Corrección, reexperimentación y comunicación	10%
Total	**100%**

7. Entregables

Repositorio del código
Configuración de fallas, scripts, instrucciones.

Evidencia organizada
baseline/, experiments/, predictions/, results/, logs/, metrics/, traces/.

Informe técnico (máx. 10 páginas)
Tabla de experimentos, resultados, análisis y aprendizajes.

8. Tabla de experimentos (obligatoria)

ID	Familia	Falla	Configuración	Predicción	Resultado	Evidencia	Aprendizaje
A1	Latencia	Tributaria lenta	+800 ms, 30%	p95 +800 ms	p99 = 12 s	métricas + logs	timeout impedia abrir breaker
B1	BD	Pool agotado	max 10 conn, 5 min	aumento de errores	503 y timeouts	métricas + trazas	ajustar pool y retry

9. Condiciones de aprobación

- Entre 4 y 8 experimentos (1–2 por familia).
- Fallas configurables y sin redespliegue.
- Kill switch funcional.
- Predicción previa documentada.
- Evidencia suficiente por experimento.
- Procedimiento reproducible.
- Análisis y aprendizaje de cada experimento.

10. Lo que NO se debe premiar

- Solo "la aplicación se cayó".
- Un throw Exception como falla.
- Gráficas sin hipótesis.
- Solo promedios de latencia.
- Muchos dashboards sin interpretación.
- Una predicción siempre acertada (su valor está en el aprendizaje).

El corazón del proyecto

Fallas configurables · Experimentos · Observabilidad · Aprendizaje



No buscamos sistemas que nunca fallen.
Buscamos arquitectos capaces de entender qué ocurre cuando fallan.